Hardware/Software Co-Design and Optimization of a MNIST CNN on Xilinx FPGAs

Christos Mountakis, AEM: 03801
Department of Electrical and Computer Engineering
University of Thessaly
Volos, Greece
Email: cmountakis@uth.gr

***Abstract*—The deployment of Convolutional Neural Networks on edge devices is heavily bottlenecked by hardware resource limitations, specifically Digital Signal Processors (DSPs) and Block RAM (BRAM), alongside strict latency requirements. In this paper, we present a comprehensive Design Space Exploration of a CNN trained on the MNIST dataset, targeted for a Xilinx FPGA. By leveraging the hls4ml compiler framework, we evaluate the complex interplay between Post-Training Quantization (PTQ) and unstructured pruning. Our exploration sweeps bit-width precision from 16-bit down to 2-bit and sparsity levels up to 90%. Furthermore, we investigate the architectural impact of the hardware Reuse Factor on synthesized logic. Our results demonstrate that unstructured sparsity yields zero logic reduction at high reuse factors due to static loop bounds, but achieves massive Look-Up Table (LUT) savings in fully unrolled (rf=1) pipelines. Our final Golden Model (4-bit precision, 70% sparsity, rf=1) achieves a 66.4x reduction in DSP utilization and a 1.49x latency speedup compared to the uncompressed baseline, while maintaining a 90.92% test accuracy.**

***Index Terms*—FPGA, CNN, hls4ml, Quantization, Pruning, Design Space Exploration, Hardware Acceleration, Edge AI.**

I. Introduction

The integration of Machine Learning algorithms into latency-critical embedded systems has driven a shift towards Field Programmable Gate Arrays (FPGAs). Unlike Graphics Processing Units (GPUs) or specialized Tensor Processing Units (TPUs) that rely on batch processing, FPGAs offer deterministic, ultra-low latency execution via deeply pipelined, spatial dataflow architectures. However, mapping a standard deep Convolutional Neural Network directly onto FPGA fabric often exceeds the available logic resources, specifically the dedicated multiplier blocks (DSPs) and distributed memory (LUTs).

To bridge this gap, hardware/software co-design techniques such as quantization and pruning have become mandatory. Quantization reduces the bit-width of activations and weights, allowing multiplication operations to be executed using standard LUTs instead of DSPs. Pruning forces neural network connections to zero, theoretically reducing the total number of Multiply-Accumulate (MAC) operations required during inference.

In this project, we utilize the hls4ml framework—an open-source compiler that translates trained Keras/TensorFlow models into High-Level Synthesis (HLS) C++ code—to perform a systematic Design Space Exploration. The primary objective is to optimize a CNN for the MNIST dataset by finding the optimal balance between hardware resource consumption and inference latency.

The design space exploration and hardware/software co-design of an ultra-low latency CNN on FPGA fabric presents several critical technical challenges, which we specifically address in this paper:

- **Precision vs. Accuracy Degradation:** Determining the lowest viable bit-width (down to 2 bits) that maintains acceptable classification accuracy using QKeras quantization mechanisms.
- **The Sparsity/Folding Paradox:** Understanding and interpreting Vivado HLS's optimization behavior when dealing with unstructured pruning. Specifically, managing the counter-intuitive reality that high sparsity only translates to physical LUT reductions in fully unrolled data paths, while folded loops inherently ignore unstructured zeros due to static scheduling constraints.
- **Resource Pareto Optimization:** Navigating the trade-off between DSP reduction, LUT consumption, and total clock cycles. The challenge lies in interpreting the synthesis reports to design a streaming architecture that optimally balances area footprint with latency requirements.

Through extensive HLS synthesis sweeps, we present a final Golden Model architecture that drastically reduces the hardware footprint, providing a blueprint for deploying ultra-low latency CNNs on resource-constrained edge FPGAs.

II. Related Work

The acceleration of neural networks on FPGA platforms has been extensively studied over the past decade. Early works primarily focused on accelerating standard floating-point operations using monolithic systolic arrays. However, as the computational demands of deep learning skyrocketed, the focus shifted toward aggressive model compression techniques.

Han et al. [4] pioneered the concept of "Deep Compression," demonstrating that neural networks could be heavily pruned and quantized without significant loss of accuracy. This inspired hardware designers to build custom FPGA accelerators capable of exploiting sparsity.

The introduction of the `hls4ml` framework by Fahim et al. [1] revolutionized the deployment of ML models for particle

physics and edge computing. By automating the translation of high-level Python models into HLS C++, `hls4ml` enabled rapid prototyping of ultra-low latency inference engines. Furthermore, the development of QKeras [2] allowed for quantization-aware training, ensuring that the software models accurately reflect the fixed-point arithmetic of the synthesized hardware.

Despite these advancements, the interplay between unstructured sparsity and hardware loop unrolling remains a complex design challenge. While prior works have demonstrated the benefits of structured pruning (removing entire filters) for FPGA deployment, our work contributes a detailed empirical analysis of how unstructured pruning behaves under varying HLS loop folding configurations.

III. Mathematical Background

A. Convolutional Neural Networks

A Convolutional Neural Network consists of multiple layers of feature extraction followed by fully connected classification layers. The core operation in a convolutional layer is the 2D discrete convolution, mathematically defined for a given output pixel $Y_{i,j}$ as:

$$Y_{i,j} = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} \sum_{c=0}^{C_{in}-1} W_{m,n,c} \cdot X_{i+m,j+n,c} + B \quad (1)$$

where X is the input feature map, W represents the kernel weights of spatial size $K \times K$, C_{in} is the number of input channels, and B is the bias term. In hardware, this operation requires $K^2 \times C_{in}$ Multiply-Accumulate (MAC) operations per output pixel. For deep networks, the total number of MACs scales exponentially, necessitating massive parallelization.

B. Magnitude-Based Unstructured Pruning

To reduce the computational burden, we employ unstructured pruning. Unstructured pruning selectively removes individual weights from the network based on their absolute magnitude. The network is trained using a Polynomial Decay schedule, which gradually increases the sparsity level S_t at training step t according to:

$$S_t = S_{final} + (S_{initial} - S_{final}) \left(1 - \frac{t - t_0}{n\Delta t}\right)^3 \quad (2)$$

where $S_{initial}$ is the starting sparsity (typically 0), S_{final} is the target sparsity (e.g., 70%), and $n\Delta t$ represents the total pruning duration. By slowly forcing low-magnitude weights to zero during training, the network is allowed to adapt and redistribute its representational capacity to the surviving weights, thereby preserving accuracy.

C. Fixed-Point Quantization

FPGAs execute fixed-point arithmetic significantly faster and with less area than floating-point arithmetic. We utilize QKeras to enforce fixed-point quantization during training. A fixed-point number can be represented as $\langle W, I \rangle$, where W is the total bit-width and I is the number of integer bits. The quantization function applied to the weights and activations is defined as:

$$Q(x) = \text{clip}\left(\text{round}\left(\frac{x}{\Delta}\right) \cdot \Delta, -2^{W-1}, 2^{W-1} - 1\right) \quad (3)$$

where $\Delta = 2^{-(W-I)}$ represents the step size or resolution of the quantizer. By varying W between 2 and 8 bits, we directly control the precision of the datapath synthesized by Vivado HLS.

IV. Proposed Methodology and Architecture

A. Dataset and Preprocessing

The models were trained and evaluated on the standard MNIST dataset, which consists of 60,000 training images and 10,000 testing images of handwritten digits. The images are 28x28 pixels in grayscale format. As part of the preprocessing pipeline, all pixel values were normalized to the range $[0, 1]$ by dividing by 255.0. This normalization is critical for fixed-point hardware, as it bounds the input dynamic range.

B. Baseline CNN Architecture

We designed a sequential CNN architecture optimized for edge deployment. The network topology was carefully sized to provide sufficient learning capacity while remaining small enough to be fully unrolled on the target FPGA. The exact layer-by-layer architecture is detailed in Table I.

TABLE I
Layer-by-Layer CNN Architecture

Layer Type	Output Shape	Parameters
Input	(28, 28, 1)	0
Conv2D (3x3)	(26, 26, 16)	160
BatchNormalization	(26, 26, 16)	64
ReLU Activation	(26, 26, 16)	0
MaxPooling2D (2x2)	(13, 13, 16)	0
Conv2D (3x3)	(11, 11, 16)	2,320
BatchNormalization	(11, 11, 16)	64
ReLU Activation	(11, 11, 16)	0
MaxPooling2D (2x2)	(5, 5, 16)	0
Conv2D (3x3)	(3, 3, 24)	3,480
BatchNormalization	(3, 3, 24)	96
ReLU Activation	(3, 3, 24)	0
MaxPooling2D (2x2)	(1, 1, 24)	0
Flatten	(24)	0
Dense	(42)	1,008
BatchNormalization	(42)	168
ReLU Activation	(42)	0
Dense	(64)	2,688
BatchNormalization	(64)	256
ReLU Activation	(64)	0
Dense (Output)	(10)	650
Total Parameters		**10,954**

C. Hardware/Software Co-Design Flow

Our Design Space Exploration follows a rigorous three-step co-design methodology:

1) **Sparsity Sweeps:** We iteratively retrain the baseline model using TensorFlow Model Optimization to target sparsity levels of 50%, 70%, and 90%. This isolates the

impact of pruning on accuracy before quantization noise is introduced.

2) **Quantization Sweeps:** The surviving sparse architectures are rebuilt using QKeras layers. We parameterize the model to sweep across bit-widths of 2, 4, 6, and 8 bits.
3) **Hardware Synthesis & Reuse Factor:** The optimal software model (the "Golden Model") is exported via `hls4ml`. We utilize Vivado HLS to synthesize the C++ code into RTL, iteratively sweeping the **Reuse Factor (RF)** from 1 to 2048 to generate a comprehensive hardware Pareto curve.

V. Results and Analysis

A. Impact of Unstructured Pruning

The first phase of our Design Space Exploration focused purely on the impact of unstructured pruning on the baseline 16-bit floating-point model. The model was retrained using the PolynomialDecay scheduling algorithm across four target sparsity levels: 0%, 50%, 70%, and 90%. The test accuracies are summarized in Table II.

TABLE II
Impact of Unstructured Pruning on 16-bit Baseline Accuracy

Sparsity Level	Test Accuracy (%)
0% (Baseline)	97.99%
50%	97.44%
70%	97.10%
90%	95.13%

As shown in Table II, the network is highly robust to unstructured pruning. At 50% and 70% sparsity, the accuracy degradation is practically negligible. Even at an aggressive 90% sparsity, the model maintains a strong 95.13% test accuracy. However, because floating-point sparsity behaves very differently in hardware when combined with quantization, we carry all three configurations (50%, 70%, and 90%) forward to the QKeras phase to observe how they compound with fixed-point arithmetic noise.

B. Quantization Sweeps (QKeras)

The network was rebuilt using QKeras layers to sweep across four bit-widths (2, 4, 6, and 8 bits) for all three sparsity targets (50%, 70%, and 90%), as well as a 0% sparsity configuration to establish a pure quantization baseline. The results of this quantization sweep are detailed in Table III.

TABLE III
Test Accuracy across Bit-Width and Sparsity Configurations

Bit-Width	0% (Pure Quantization)	50% Sparsity	70% Sparsity	90% Sparsity
8-bit	95.79%	95.80%	96.25%	82.70%
6-bit	95.85%	95.23%	93.01%	86.24%
4-bit	92.44%	91.73%	90.92%	84.14%
2-bit	88.88%	74.60%	73.57%	65.86%

The quantization sweep reveals several critical thresholds. Because MNIST is a relatively simple dataset to predict, we established a strict acceptability threshold of $> 90\%$ accuracy for edge deployment.

As shown in the empirical data, the 90% sparsity model failed to meet this threshold across all bit-widths, peaking at only 86.24% and degrading to 84.14% at 4-bit precision. This objectively proves that while the network can handle heavy pruning in floating-point, compounding 90% sparsity with fixed-point quantization noise results in an unacceptable loss of representational capacity.

To evaluate the true impact of Hardware/Software Co-Design, we compared these heavily pruned models against a **Pure Quantization (0% Sparsity)** baseline. Although the 0% sparse models achieved slightly higher absolute accuracies (e.g., 92.44% at 4-bits), they require significantly more physical LUTs to implement all unrolled multipliers in hardware.

By employing Co-Design, the 4-bit configuration at 70% sparsity presents the most optimal choice. It drops marginally to 90.92% (successfully completing our strict accuracy threshold), but the 70% unstructured sparsity yields massive logic savings in the hardware datapath. Consequently, the **4-bit, 70% sparse** configuration was selected as our **Golden Model**.

C. Memory Footprint Compression

An often overlooked aspect of FPGA deployment is the limitation of on-chip Block RAM. By tracking the parameter count across our Design Space Exploration, we can quantify the exact memory compression achieved by our Golden Model.

The baseline architecture contains 10,954 trainable parameters. In the baseline 16-bit configuration, the total memory footprint for the weights is:

$$\begin{aligned} \text{Baseline Size} &= 10,954 \times 16 \text{ bits} \\ &= 175,264 \text{ bits} \approx 21.9 \text{ KB} \end{aligned} \tag{4}$$

Our Golden Model aggressively reduces this through both 70% unstructured pruning (which mathematically zeroes out 70% of the weights, meaning they do not need to be stored in logic) and 4-bit quantization. The active memory footprint becomes:

$$\begin{aligned} \text{Active Params} &= 10,954 \times (1 - 0.70) \\ &\approx 3,286 \text{ parameters} \end{aligned} \tag{5}$$

$$\begin{aligned} \text{Golden Size} &= 3,286 \times 4 \text{ bits} \\ &= 13,144 \text{ bits} \approx 1.64 \text{ KB} \end{aligned} \tag{6}$$

This represents a **13.3x reduction in memory footprint**, shrinking the model from 21.9 KB down to a mere 1.64 KB. This massive compression is what allows the `hls4ml` compiler to map the entire network's weights directly into distributed LUT logic rather than relying on slower, scarce BRAM blocks, significantly accelerating inference latency.

D. The Sparsity / Folding Paradox

A major architectural discovery during the synthesis phase was a fundamental limitation in how hls4ml and Vivado handle unstructured sparsity.

When the Reuse Factor is set to 1 (fully unrolled), hls4ml physically instantiates every MAC operation in parallel logic.

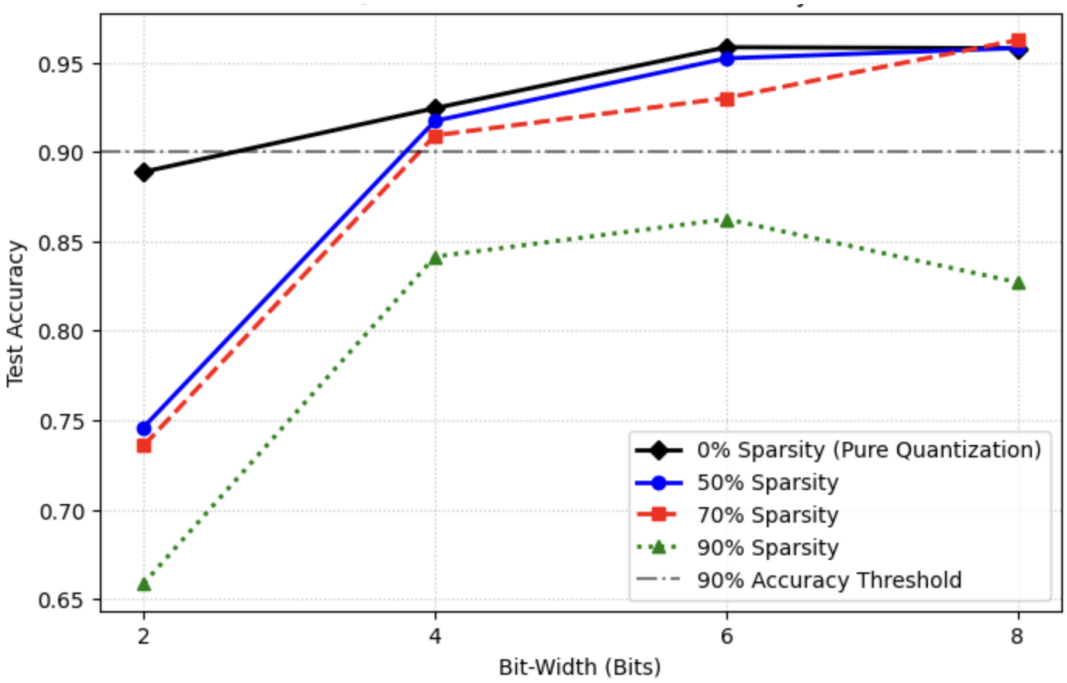


Fig. 1. Quantization Bit-Width vs. Test Accuracy across 0%, 50%, 70%, and 90% Sparse Models.

During synthesis, Vivado detects static multiplications by zero (the pruned weights) and aggressively optimizes those logic gates away. This results in massive LUT savings.

However, when the Reuse Factor is increased (e.g., RF=16 or RF=64), the architecture is "folded" into a sequential loop that iterates over the MAC operations to save area by reusing DSPs and multipliers. Because C++ loop bounds in HLS must be statically determined during synthesis, the hardware must execute every single cycle of the loop, even if the weight fetched from memory on a given cycle is exactly zero. The hardware cannot dynamically skip operations in a pipeline without introducing complex control logic that would severely violate timing constraints.

Consequently, we formulate the **Sparsity/Folding Paradox**: unstructured sparsity yields zero logic area reduction at high reuse factors. The sparsity only manifests as physical logic reduction in fully unrolled data paths.

E. Pareto Optimization and DSE Summary

To evaluate the final hardware trade-offs, we generated Vivado synthesis reports for multiple configurations of our Golden Model, compared against the 16-bit uncompressed baseline. The results are summarized in Table IV.

Key Observations:

- **Fastest Implementation:** The Golden Model at RF=1 achieved a **1.49x Speedup** (from 2371 to 1588 cycles) over the baseline while using **66.4x fewer DSPs**. The LUT count was reduced by 1.8x (from 295k to 163k) directly as a result of the 70% unstructured sparsity successfully optimizing away unrolled multipliers.

TABLE IV
DESIGN SPACE EXPLORATION RESULTS

Configuration	Latency (Cycles)	DSPs	LUTs	FFs
Baseline (16-bit, 0% Sparse, rf=1)	2371	7040	295890	113055
Golden Model (4-bit, 70% Sparse, rf=1)	1588	106	163496	37923
Golden Model (rf=4)	7094	539	132785	58924
Golden Model (rf=16)	18831	135	71410	45015
Golden Model (rf=64)	61168	2	57256	43163
Golden Model (rf=256)	117760	2	54193	42473
Golden Model (rf=1024)	195887	2	52679	43489
Golden Model (rf=2048)	390398	2	54199	44504

- **Smallest Implementation:** The Golden Model at RF=1024 crushed DSP utilization down to the absolute theoretical streaming minimum of 2 DSPs (a 3520x reduction), reducing LUTs by 5.6x (down to 52,679), but at the cost of an 82.6x latency slowdown. Increasing the RF further to 2048 yielded no further area improvements, proving that LUT usage plateaued around ∼52k as sparsity no longer provided area savings in deeply folded loops.

F. CPU Baseline Comparison

To demonstrate the real-world value of this hardware acceleration, we benchmarked the Golden Model against a standard desktop CPU environment using TensorFlow. In latency-critical edge AI applications (such as autonomous navigation or sensor fusion), data arrives in a continuous stream rather than in bulk. Therefore, the system must first be evaluated at a batch size of 1.

Running the quantized Golden Model on the CPU at `batch_size=1` yielded an average inference latency of 691.67 μs per image. In contrast, the fully unrolled FPGA

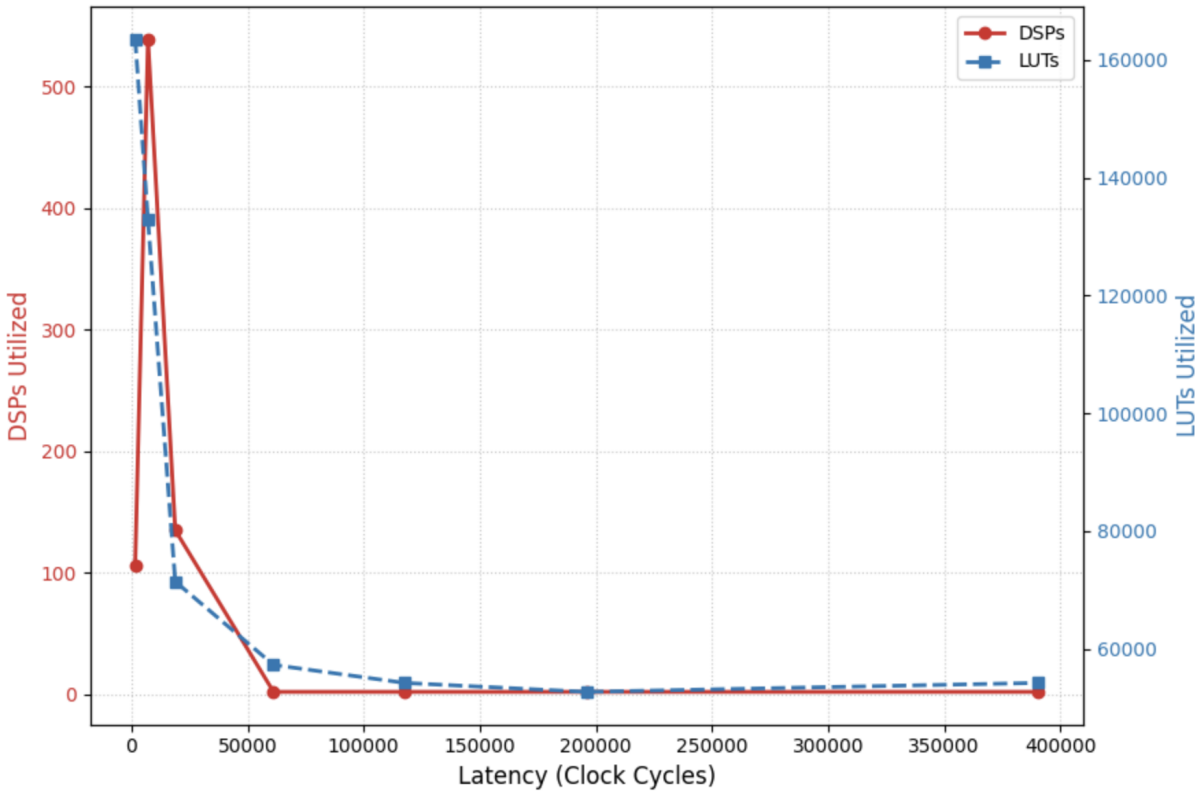


Fig. 2. Dual-Axis Pareto Curve illustrating the trade-off between Hardware Resources (DSPs and LUTs) and Latency across varying Reuse Factors.

Golden Model requires exactly 1,588 clock cycles. Assuming a standard 200 MHz (5 ns) clock constraint, the FPGA achieves a deterministic latency of just 7.94 μs per image. This represents an **87.1x latency speedup** over the CPU baseline.

To determine if the CPU could eventually overtake the FPGA through bulk parallelization (throughput), we aggressively swept the CPU batch size from 1 to 10,000. While deep vectorized CPU instructions improved performance at larger batches, the CPU's processing time plateaued at approximately 28.07 μs per image (at `batch_size=10000`) due to cache thrashing and memory bottlenecks. Remarkably, the CPU never achieved parity. The FPGA's streaming latency of 7.94 μs remained **3.5x faster than the CPU's absolute maximum bulk throughput**.

This conclusively proves that the FPGA's spatial architecture, which flows uninterrupted through physical logic gates without instruction fetching overhead, dominates standard processors in both latency and throughput for edge inference.

VI. Conclusion

This work presented a comprehensive Design Space Exploration of a Convolutional Neural Network deployed on FPGA hardware using the hls4ml framework. By heavily leveraging QKeras quantization and unstructured pruning, we mapped out a vast architectural trade-off space.

Our analysis highlighted a critical architectural nuance: unstructured sparsity only translates to hardware savings in fully unrolled (RF=1) architectures. For folded architectures (RF¿1), the fixed nature of HLS loop bounds negates the benefits of unstructured pruning. Ultimately, the flexibility of the Reuse Factor parameter allowed us to plot a definitive Pareto curve, proving that a 4-bit, 70% sparse architecture can drastically outperform an uncompressed baseline in hardware efficiency, providing a clear methodology for scaling neural network inference across varying FPGA resource constraints.

References

[1] F. Fahim *et al.*, "hls4ml: An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices," *Frontiers in Big Data*, vol. 4, p. 703396, 2021, doi: 10.3389/fdata.2021.703396.

[2] C. Coelho *et al.*, "Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors," *Nature Machine Intelligence*, vol. 3, pp. 675–686, 2021.

[3] Xilinx Inc., "Vitis High-Level Synthesis User Guide (UG1399)," *AMD Documentation*, 2023.

[4] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," in *International Conference on Learning Representations (ICLR)*, 2016.